Focus Frontier: A Neurodiversity-First Cognitive Training and Clinical Assessment Platform with Adaptive Difficulty and Real-Time Multiplayer

Manvendra Rai
Computer Science & Engineering
(Gaming Technology)
VIT Bhopal University
Bhopal, India
manvendrarai2022@Vitbhopal.ac.in

Sahaj Singh
Computer Science & Engineering
(Gaming Technology)
VIT Bhopal University
Bhopal, India
sahajsingh2022@vitbhopal.ac.in

Raman Kumar
Computer Science & Engineering
(Gaming Technology)
VIT Bhopal University
Bhopal, India
ramankumar2022@vitbhopal.ac.in

Triven Khoushi
Computer Science & Engineering
(Gaming Technology)
VIT Bhopal University
Bhopal, India
trivenkhoushi2022@vitbhopal.ac.in

Rishi Bhargava
Computer Science & Engineering
(Gaming Technology)
VIT Bhopal University
Bhopal, India
rishibhargava2022@vitbhopal.ac.in

***Abstract*—Focus Frontier is a browser-based cognitive training and clinical assessment platform that addresses two shortcomings common to existing brain-training tools: static difficulty that never adjusts to user performance, and designs that assume a neurotypical user. Built on a MongoDB-Express-Node.js stack with a Vanilla JavaScript frontend, the system delivers nine scientifically grounded activities—six adaptive training games and three digitised neuropsychological assessments—covering six cognitive domains: working memory, sustained attention, processing speed, cognitive flexibility, inhibitory control, and reaction time. A three-component server-side adaptive engine uses heuristic algorithms grounded in Vygotsky's Zone of Proximal Development to normalise session scores and adjust difficulty toward a 70% target success rate. The platform additionally supports real-time 1v1 multiplayer matches, where a Linear Congruential Generator (LCG) seeded randomness guarantee ensures both players face identical game states. A cross-game skill-bucket analytics system aggregates performance across 14 cognitive skill tags and surfaces targeted improvement recommendations. Across 121 recorded sessions the system maintained a median API response time below 150 ms and sustained 60 FPS gameplay. WCAG 2.1 Level AA accessibility is met across all nine activities, with user accessibility preferences persisted at the database level rather than locally.**

***Index Terms*—adaptive difficulty, cognitive training, neurodiversity, neuropsychological assessment, heuristic machine learning, real-time multiplayer, seeded randomness, web application, gamification, inclusive design, Zone of Proximal Development.**

I. I. INTRODUCTION

Cognitive decline and neurodevelopmental conditions affect a substantial portion of the global population. People with ADHD, autism spectrum conditions, or dyslexia stand to gain considerably from regular, structured cognitive practice, yet the tools available to them are largely built around an assumed neurotypical user. The irony is that the very population most likely to benefit from brain-training software is also the population least well served by what currently exists.

Two structural problems recur across mainstream platforms. First, difficulty is typically fixed at design time. Once a user has grown beyond the preset challenge level, the training effect plateaus and the activity becomes repetition rather than practice. Second, clinical neuropsychological assessments—the instruments that psychologists actually use to measure working memory, inhibitory control, and processing speed—exist in a separate world from

consumer apps. There is no widely available tool that combines both in a single, browser-accessible interface.

Focus Frontier was built to address both problems simultaneously. The platform delivers nine scientifically grounded activities: six adaptive cognitive training games and three digitised versions of established clinical assessments, specifically the Dual N-Back [2], Go/No-Go task [10], and Trail Making Test [11]. A three-component adaptive engine reads each user's session history and adjusts difficulty toward a 70% target success rate, a threshold grounded in Vygotsky's Zone of Proximal Development [5]. On top of this, the system includes real-time 1v1 multiplayer with cryptographically seeded randomness to guarantee fairness, a cross-game cognitive profiling dashboard, and database-level accessibility preferences for neurodivergent users.

A. *A. Research Objectives*

The goals driving this project were: (1) design a cognitive training platform where neurodiversity accommodation is an architectural constraint, not a retrofit; (2) integrate established clinical assessment tasks into the same environment as training games, removing the need for separate tools; (3) implement a server-side adaptive difficulty engine that is transparent and explainable rather than a black-box model; (4) build a real-time multiplayer system in which both players face provably identical game states; (5) construct a cross-game skill analytics pipeline that identifies specific cognitive weaknesses and recommends targeted training.

B. *B. Novel Contributions*

To the authors' knowledge this is the first web platform to combine casual cognitive training games with digitised clinical neuropsychological assessments under one authentication system. The explainable heuristic adaptive engine, the LCG-based seeded-random multiplayer, the per-game ML specialisation demonstrated by TrailMakingModel, and the 14-bucket cross-game cognitive profiling system are each individually novel. Section XII discusses these contributions in detail.

C. *C. Paper Organisation*

Section II reviews the research literature. Section III describes the overall system architecture. Sections IV through VIII cover the game modules, adaptive engine, multiplayer system, analytics pipeline, and accessibility design respectively. Section IX details implementation decisions around the database schema and REST API. Section X presents testing results. Section XI discusses novel contributions, and Section XII concludes with directions for future work.

II. II. LITERATURE REVIEW

A. *A. Cognitive Training and Transfer Effects*

Klingberg and colleagues [1] provided some of the earliest controlled evidence that adaptive working memory training produces measurable improvements in children with ADHD, a finding that motivated a generation of subsequent work. Jaeggi et al. [2] showed that sustained Dual N-Back practice can improve fluid intelligence, though subsequent meta-analyses have been more cautious about the size and durability of these gains. The large-scale study by Owen et al. [4] involving more than 11,000 participants found no evidence of far-transfer from trained tasks to untrained cognitive domains, a result that any honest platform description must acknowledge. The working consensus in the field is that within-domain improvements are achievable when training is adaptive and sustained, but broad claims about general intelligence enhancement are not well supported.

B. *B. Adaptive Difficulty and the Zone of Proximal Development*

Vygotsky's concept of the Zone of Proximal Development [5] holds that learning is maximised when a task sits just above a learner's current independent capability. This theoretical anchor translates directly into a practical design target: a task that a user succeeds at roughly 70% of the time sits in the productive zone. Csikszentmihalyi's flow theory [6] reaches a complementary conclusion from a motivational direction—engagement is highest when challenge and skill are approximately matched. Lomas et al. [7] demonstrated empirically that games which maintain this balance show measurably better learning outcomes and higher session retention than those with static difficulty.

C. *C. Gamification in Cognitive Training*

Deterding et al. [8] defined gamification as the application of game design elements in non-game contexts and identified four conditions for effectiveness: mechanics must serve the learning goal, difficulty must sustain motivation, feedback must be fast and informative, and users must retain a sense of agency. Hamari, Koivisto, and Sarsa [13]

conducted a systematic review of empirical gamification studies and found significant positive effects on user engagement, though effect sizes varied considerably by context and task type. Focus Frontier applies all four of Deterding's conditions across its game layer.

D. *D. Neuropsychological Assessment Tasks*

The three clinical tasks incorporated in Focus Frontier each have decades of validation behind them. The Dual N-Back is described by Jaeggi et al. [2] as a gold-standard measure of working memory capacity in cognitive neuroscience. The Go/No-Go paradigm is a standard instrument for assessing response inhibition and impulsivity, and it appears routinely in ADHD evaluation batteries [10]. Stroop [15] demonstrated in 1935 that the time required to name the ink colour of a colour-word increases dramatically when the word and ink colour conflict, revealing automatic versus controlled processing; the Color Cascade game draws on this interference paradigm. The Trail Making Test [11] was first validated by Reitan as a measure of visual attention and task-switching; Part A (sequential numbers) reflects processing speed, while Part B (alternating numbers and letters) indexes cognitive flexibility.

E. *E. Neurodiversity and Inclusive Design*

Singer [9] coined the term neurodiversity and argued that conditions such as ADHD and autism represent natural variation in human cognition rather than deficits requiring correction. This framing has practical consequences for platform design: accessible features should be built in from the start, not added as an afterthought. WCAG 2.1 [14] formalises this into testable criteria. Mishra, Anguera, and Gazzaley [16] showed through the NeuroRacer study that cognitive training designed specifically for a target population outperforms generic training, providing empirical backing for population-specific design.

F. *F. Research Gap*

Despite the substantial literature on both cognitive training and adaptive game design, no widely available consumer platform has integrated clinical neuropsychological assessments alongside adaptive training games under a single interface with explicit neurodiversity-first design. Most adaptive systems use rule-based thresholds but describe them using ML terminology, which conflates fundamentally different approaches and makes clinical interpretation harder. Focus Frontier addresses both gaps directly.

III. III. SYSTEM ARCHITECTURE

A. *A. Architectural Overview*

Focus Frontier follows a three-tier architecture. The browser-based client handles rendering and user interaction. The Express.js API layer manages authentication, session tracking, and the adaptive difficulty pipeline. MongoDB at the data tier stores all persistent records. Each tier can be modified or scaled independently. Communication between client and server happens over REST for data operations and Socket.IO WebSocket for real-time multiplayer events.

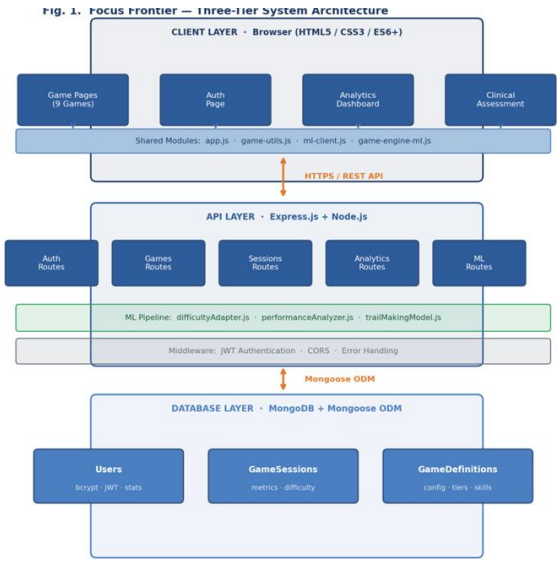

Fig. 1. Focus Frontier High-Level System Architecture.

B. *B. Hybrid Client Architecture*

The client is split into two distinct zones. Core shared libraries under client/assets/ handle authentication, API communication, ML client integration, game utilities, and styling. Each of the nine games, however, is a fully self-contained HTML file with its own embedded JavaScript game loop. This Hybrid Client Architecture was a deliberate engineering choice driven by three requirements: zero framework overhead during gameplay (no React or Vue virtual DOM diffing competes with the rendering loop), stable 60 FPS achieved through direct DOM manipulation and requestAnimationFrame, and millisecond-precision input handling critical for valid reaction-time measurement. Each game also has its own *-ml.js adapter file that customises how ML recommendations are consumed and displayed for that specific game.

This architecture separates the portal experience—hub page, analytics, authentication—from the measurement experience in the game engines, so performance

optimisations in either zone do not interfere with the other.

C. *C. Technology Stack*

TABLE I: TECHNOLOGY STACK

Layer	Technology	Rationale
Frontend	Vanilla JavaScript (ES6+)	Zero-overhead game rendering; no framework bottleneck
Styling	CSS3 Variables + Grid + Glassmorphism	Modern, responsive, accessibility-ready design
Charts	Chart.js	Interactive analytics visualisations
Backend	Node.js + Express.js	High-throughput REST API
Real-time	Socket.IO	WebSocket multiplayer with HTTP long-polling fallback
Database	MongoDB + Mongoose	Flexible schema for heterogeneous session data
Auth	JWT + bcryptjs	Stateless auth with 7-day token expiry
ML	Heuristic-based JS modules	Lightweight, explainable, no Python dependency

D. *D. Server Components*

The server exposes five REST route groups (auth, games, sessions, analytics, ML) plus a Socket.IO handler for multiplayer. A single JWT middleware function guards all private routes by decoding the token, extracting the user ID, and attaching it to the request context; individual route handlers need not repeat this check. The ML engine runs as three pure-JavaScript server-side modules, avoiding any Python or TensorFlow dependency. The multiplayer manager maintains an in-memory Map of active game rooms, separate from MongoDB, to avoid database round-trips on every real-time event.

E. *E. Database Collections*

MongoDB holds four collections. Users stores bcrypt-hashed credentials (10 salt rounds), display names, and per-user accessibility preferences. GameSessions stores one document per play session with a flexible metrics array to accommodate heterogeneous data across nine different games without requiring schema changes. GameDefinitions holds configuration for each game including cognitive skill tags, difficulty tier parameters, and scoring rules. Matches stores the outcome of each 1v1 multiplayer duel with host and guest scores, winner determination, and game settings.

IV. IV. GAME MODULE DESIGN

A. *A. Design Philosophy*

Every game was designed around a specific cognitive target drawn from established neuropsychology research. The six training games offer four difficulty tiers (Easy, Medium, Hard, Expert); changing tier is not cosmetic—it modifies the underlying mechanics, grid sizes, timing windows, object speeds, and scoring multipliers. The three clinical assessment tasks use continuously configurable parameters rather than fixed tiers, reflecting how they are administered in actual clinical practice. Table II lists the cognitive science basis for each of the nine games.

TABLE II: COGNITIVE SCIENCE BASIS FOR ALL NINE GAMES

Game	Research Basis
Memory Matrix	Corsi Block-Tapping Task variants
Focus Sphere	Continuous Performance Test (CPT) paradigm
Reflex Runner	Eriksen Flanker Task — inhibition and reaction time
Color Cascade	Stroop Effect (Stroop, 1935) [15]
Pattern Path	Wisconsin Card Sort Test concepts
Shape Sorter	Processing speed / categorisation, WAIS-IV subtests
Dual N-Back	Jaeggi et al. (2008) — working memory capacity [2]
Go/No-Go Task	Standard ADHD / executive function assessment [10]
Trail Making Test	Reitan (1958) — visual attention and task-switching [11]

B. *B. Cognitive Training Suite*

Memory Matrix targets working memory, spatial recall, and pattern recognition. Players flip pairs of face-down cards on a grid to find matches. Grid size scales from 3×3 (9 cards, 2000 ms per card, 100 max moves) at Easy up to 6×6 (36 cards, 500 ms per card, 40 max moves) at Expert. Scoring uses a base match value of 10 to 25 points plus a

time bonus that decreases from 100 to 25 as difficulty rises.

Focus Sphere measures sustained attention and vigilance. Targets spawn at timed intervals and must be clicked before they disappear; the sphere size shrinks from 80 px at Easy to 30 px at Expert, and spawn rate tightens from 2000 ms to 500 ms. Each hit scores 10 to 25 points; each miss incurs a penalty of 5 to 15 points. The game duration increases from 60 s to 120 s at higher difficulties, placing greater demands on sustained concentration.

Reflex Runner is a side-scrolling obstacle avoidance game targeting inhibitory control, reaction time, and hand-eye coordination. Obstacle spawn rate compresses from 1500 ms to 400 ms and obstacle speed rises from 3 to 9 units. Player speed increases from 5 to 8 to compensate partially. Power-up frequency rises from every 8 s to every 2 s at Expert, rewarding consistent performance.

Color Cascade draws on the Stroop interference paradigm. Coloured objects fall and must be sorted into matching buckets under time pressure; drop rate increases from 1500 ms to 400 ms across difficulty tiers and colour variety expands from 3 to 6 colours. A combo multiplier unique to this game scales from 1.1× at Easy to 1.5× at Expert, providing an exponential reward for consecutive correct sorts that amplifies engagement.

Pattern Path targets visual planning, cognitive flexibility, and spatial awareness. The system shows a path across a grid and the player must reproduce it from memory. Grid size grows from 5×5 to 8×8, path length extends from 3 to 10 nodes, display time compresses from 2000 ms to 750 ms, and round count increases from 10 to 25.

Shape Sorter targets processing speed and rapid categorisation. Shapes appear on screen and must be drag-sorted into the correct category before a window closes. Shape variety increases from 3 to 6, category count from 3 to 6, spawn rate from 2000 ms to 700 ms, and the drag duration window tightens from 1500 ms to 600 ms.

C. *C. Clinical Assessment Suite*

The Dual N-Back presents simultaneous position and audio stimuli; the player must identify when the current stimulus matches the one N steps back. The N-back level varies from 1 to 4, session duration from 60 s to 80 s, and stimulus interval from 2000 ms to 1100 ms. These parameters mirror the experimental conditions used in peer-reviewed cognitive research [2].

The Go/No-Go task presents stimuli in two categories. Players must respond to Go stimuli and withhold response on No-Go stimuli. At Easy the Go ratio is 80% (few inhibition trials); at Expert it drops to 50%, placing equal demand on response and inhibition. Stimulus duration shortens from 1000 ms to 700 ms. The system separately tracks commission errors (false alarms on No-Go) and omission errors (missed Go signals), both of which are clinically relevant markers in ADHD assessment [10].

The Trail Making Test replicates Parts A and B of the standard neuropsychological instrument. Part A requires connecting numbered nodes in sequence; Part B requires alternating between numbers and letters. Node count ranges from 12 to 30, minimum inter-node distance from 80 px to 55 px, node radius from 22 px to 16 px, and the allowed time per node from 4 s to 2 s. Rather than applying a flat difficulty tier, this game uses the dedicated TrailMakingModel described in Section V.

TABLE III: TRAINING GAME DIFFICULTY PARAMETERS

Game	Easy	Expert
Memory Matrix	3×3 grid, 2000ms/card	6×6 grid, 500ms/card
Focus Sphere	80px, 2000ms spawn, 60s	30px, 500ms spawn, 120s
Reflex Runner	Spawn 1500ms, speed 3	Spawn 400ms, speed 9
Color Cascade	3 colours, 1500ms drop	6 colours, 400ms drop, 1.5× combo
Pattern Path	5×5, 3 nodes, 2000ms	8×8, 10 nodes, 750ms
Shape Sorter	3 shapes, 2000ms spawn	6 shapes, 700ms spawn, 600ms drag

V. V. ML-BASED ADAPTIVE DIFFICULTY ENGINE

A. *A. Design Philosophy and Terminology*

The adaptive engine is a heuristic rule-based system implemented in pure JavaScript. The decision to avoid deep learning was intentional: session volumes in any single deployment are insufficient to train a reliable model, and transparent heuristics are easier to explain to clinicians and educators who need to understand why a difficulty change was recommended. The

system is referred to as an adaptive engine rather than a machine learning model in this paper to avoid overclaiming.

B. *B. Three-Component Architecture*

The engine is split across three server-side modules and one client-side bridge. DifficultyAdapter (difficultyAdapter.js) ingests recent session history and produces a difficulty recommendation with an associated confidence score. PerformanceAnalyzer (performanceAnalyzer.js) identifies cognitive skill gaps and generates trend-based improvement reports. TrailMakingModel (trailMakingModel.js) is a game-specific adaptive module for the Trail Making Test that produces a complete spatial layout configuration rather than a simple difficulty label. MLClient (ml-client.js) is the client-side bridge that connects each game frontend to these three server modules through five dedicated API endpoints.

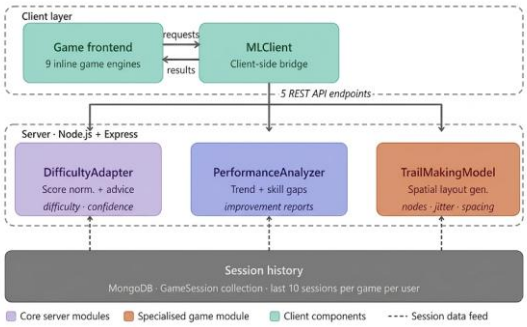

Fig. 2. Adaptive Difficulty Engine Architecture and Data Flow.

C. *C. DifficultyAdapter Algorithm*

The algorithm takes as input the last ten game sessions for a given user and game key, plus the current difficulty level. Performance is computed by summing scores across those sessions and normalising to the range [0, 1], assuming an approximate maximum of 1000 points per session: performance = min(totalScore / (sessionCount × 1000), 1.0). When no sessions exist the system defaults to 0.5, placing a new user at a medium baseline.

The target performance is 0.7, chosen to reflect a 70% success rate consistent with the Zone of Proximal Development. A tolerance band of ±0.1 creates three decision zones: if performance exceeds 0.8 the next difficulty tier is recommended (player is excelling); if performance falls below 0.6 the previous tier is recommended (player is struggling); otherwise the current tier is maintained. Confidence is computed as min(|performance − 0.7| / 0.3, 1.0) and the autoAdjust flag is set when confidence exceeds 0.7, triggering automatic application without requiring user confirmation. Below that threshold the player sees a recommendation widget with Accept and Dismiss buttons.

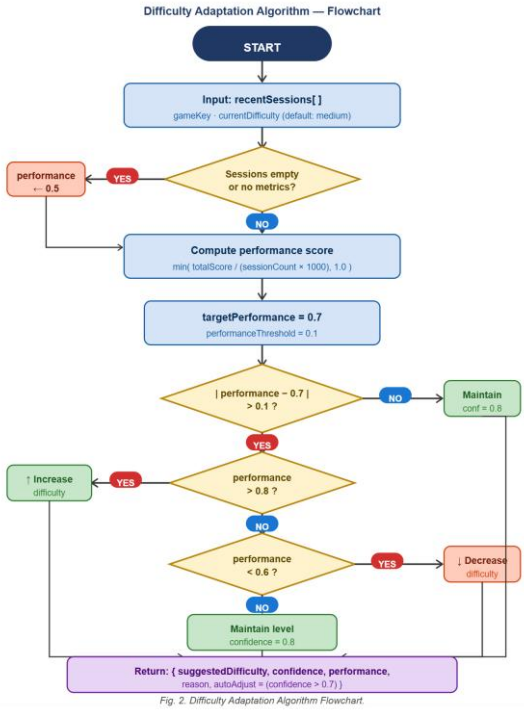

Fig. 3. Difficulty Adaptation Algorithm Flowchart.

D. *D. PerformanceAnalyzer*

PerformanceAnalyzer runs three functions. The trend analysis function computes improvement rate as ((lastScore − firstScore) / firstScore) × 100. A change above +5% is classified as improving; below −5% as declining; otherwise stable. Alongside this, consistency is measured using the inverse coefficient of variation: consistency = max(0, 1 − (σ / μ)), where higher values indicate more predictable performance.

The skill gap identification function maps each game to three cognitive skill tags. Priority levels are assigned based on trend direction and consistency: a declining trend earns high priority, consistency below 0.5 earns medium priority, and an improving trend earns low priority. The function then queries all other games sharing the same skill tag and returns cross-training recommendations—for example, if reaction-time is weak in Focus Sphere, the system recommends Reflex Runner and Go/No-Go.

The third function, generateImprovementReport, combines trend analysis and skill gaps into a unified output containing the last score, trend direction,

percentage improvement, consistency score, prioritised skill improvements, and cross-game recommendations. This report is returned to the client after every session and rendered by MLClient.

E. *E. TrailMakingModel*

The Trail Making Test receives specialised treatment because its performance depends heavily on the spatial layout of nodes, not just a flat difficulty setting. TrailMakingModel extracts three metrics from past sessions: average score, average error count, and average completion time. Based on these, it generates a complete layout configuration.

Node count starts at a base of 12 to 25 and is adjusted dynamically: high performance (≥85%) combined with fast completion adds 3 nodes; low performance (≤45%) subtracts 3 nodes. The final value is clamped to [10, 30]. Layout mode switches from grid (structured placement) to scatter (random placement) for harder configurations. A jitter parameter increases spatial randomness at higher difficulties, and minimum inter-node distance is adjusted based on error rate—more errors produce wider spacing so the visual scanning task is somewhat easier. The result is that no two Trail Making sessions are spatially identical, and the layout adapts to each player's specific pattern of errors.

F. *F. Game Engine ML Lifecycle*

The GameEngine base class in game-engine-ml.js implements a consistent ML integration lifecycle for all nine games. Before the game starts, initializeGame() creates a session record and fetches a difficulty recommendation from the server. If ML confidence exceeds 70%, the recommended difficulty is applied automatically and the player is notified. If confidence is below that threshold, the recommendation widget is shown and the player decides. During play, score, moves, accuracy, and time are tracked in real time. When the game ends, endGame() saves the session metrics, sends the data to POST /api/ml/session-complete for analysis, and displays the AI insight report.

Score calculation across all games follows the formula: finalScore = basePoints × difficultyMultiplier × accuracy, where multipliers are 1.0× (Easy), 1.5× (Medium), 2.0× (Hard), and 2.5× (Expert). This ensures that harder difficulty attempts are meaningfully rewarded even at the same raw accuracy level.

VI. VI. REAL-TIME MULTIPLAYER SYSTEM

A. *A. Architecture*

The multiplayer system is built on Socket.IO, which provides WebSocket transport with automatic HTTP long-polling fallback for environments where WebSocket connections are restricted. The server component (multiplayer.js) handles room management, player state, and score synchronisation. The client hub (multiplayer.html) is the lobby UI where players create or join rooms. The game overlay (multiplayer-game.js) is a transparent layer that is injected over any existing single-player game page.

B. *B. Room State Machine*

Each room progresses through five states: waiting (host created, no guest), selecting (host is choosing a game), ready (both players confirmed), playing (match in progress), and finished (results displayed). When the host chooses to rematch, the room resets to waiting without being destroyed. Room codes are six-character alphanumeric strings excluding visually ambiguous characters (O, 0, I, 1). Rooms are held in a server-side in-memory Map rather than MongoDB to avoid database overhead on every real-time event. A background cleanup interval runs every 60 seconds and removes rooms that have been inactive for more than 30 minutes.

Fig. 4. Multiplayer Room State Machine.

C. *C. Seeded Randomness for Fairness*

The most distinctive aspect of the multiplayer implementation is the fairness guarantee. Before the game starts, the server generates a random seed and broadcasts it to both players as part of the game_start event. The multiplayer overlay then replaces Math.random() with a deterministic Linear Congruential Generator (LCG):

seed = (seed × 9301 + 49297) % 233280; result = seed / 233280

Because both players receive the same seed and the same LCG, every obstacle position, target location, and card layout in the match is identical for both. The duel therefore measures only skill, not

luck. No network synchronisation of game state is needed for the game logic itself; only score updates are transmitted over the socket.

D. *D. Score Synchronisation*

The multiplayer overlay reads the player's current score from the DOM (looking for elements with id game-score or score) every 500 milliseconds. Each detected change is emitted to the server as a score_update event, which the server immediately forwards to the opponent as opponent_score. A fixed overlay bar at the top of the screen shows "You vs Opponent" scores in real time. Game end detection works by intercepting the closeGameWindow() function to capture the final score before the game_finished event is sent.

E. *E. Overlay Pattern and Reconnection*

Multiplayer mode is activated by URL parameters (?mp=1&room=XXXX&role=host&seed=12345&difficulty=intermediate) rather than by modifying any game's source code. The overlay injects three UI elements: an opponent score bar, a ready button, and a duel results screen. Host game settings are automatically applied to the game's difficulty dropdown. This non-intrusive overlay pattern means that adding multiplayer support to a new game requires zero changes to the game's own code.

Because a multiplayer match involves navigating to a separate game page, disconnect handling is important. The server does not destroy a room when a socket disconnects; it waits for reconnection. The game page sends a rejoin_game_room event on load, which restores the player's place in the match. Match results are persisted to MongoDB via the Match model with host and guest scores, winner, game key, settings, and timestamp.

TABLE IV: SOCKET.IO EVENTS

Event	Direction	Purpose
create_room	Client → Server	Host creates room
join_room	Client → Server	Guest joins room
player_joined	Server → Host	Notify host of guest arrival
select_game	Host → Server	Host picks the game
game_selected	Server → Both	Broadcast game choice + seed
player_ready	Client → Server	Signal readiness
game_start	Server → Both	Both ready, begin countdown
score_update	Client → Server	Real-time score broadcast
opponent_score	Server → Opponent	Forward opponent score
game_finished	Client → Server	Player completed game
game_results	Server → Both	Final results when both done
rejoin_game_room	Client → Server	Reconnect after page navigation
opponent_disconnected	Server → Player	Handle disconnection

VII. VII. ANALYTICS AND COGNITIVE PROFILING

A. *A. Skill-Bucket Aggregation System*

Each game maps to three cognitive skill tags. The analytics engine aggregates session scores across all games for each skill tag, producing a 14-bucket cognitive profile. Skills with the lowest average scores are ranked first and flagged with actionable improvement advice. Because each skill appears in multiple games, a weakness in reaction-time that shows up in Focus Sphere may be corroborated by data from Reflex Runner and Go/No-Go, making the profile more reliable than a single-game assessment.

TABLE V: THE 14 COGNITIVE SKILL BUCKETS

Skill Bucket	Games Tracking This Skill
sustained-attention	Focus Sphere
reaction-time	Focus Sphere, Reflex Runner, Go/No-Go
concentration	Focus Sphere
hand-eye-coordination	Reflex Runner
quick-decision	Reflex Runner, Color Cascade
working-memory	Memory Matrix, Dual N-Back
pattern-recognition	Memory Matrix, Pattern Path
focus	Memory Matrix
planning / spatial-awareness	Pattern Path
color-discrimination	Color Cascade
visual-processing	Color Cascade, Shape Sorter
shape-recognition / categorization	Shape Sorter

inhibitory-control / impulse-control	Go/No-Go
processing-speed / cognitive-flexibility	Trail Making Test

B. *B. Analytics API and Dashboard*

The analytics endpoint (GET /api/analytics/overview) accepts two query parameters: scope (me for personal data, all for global) and days (1 to 180, setting the lookback window). The aggregation pipeline computes per-game statistics (play count, best score, average score, average duration), a daily average score time series for Chart.js line graphs, and the skill-bucket rankings. The bottom two performing skill buckets are automatically flagged with improvement recommendations.

The client-side analytics dashboard in analytics.html visualises this data through Chart.js charts: a line graph of score trends over time, a radar chart of the six primary cognitive domains, and per-game performance bars. Users can toggle between personal and global scope and select time windows of 7, 14, 30, or 90 days.

C. *C. Local Performance Cache*

Alongside server-side analytics, each game maintains a LocalStorage-based performance cache through the GameStats class. This tracks total plays, high score, total score, best streak, and total time for each game without requiring an API call. The cache enables instant display of personal bests at game load and triggers celebration animations when a new high score is achieved. The PerformanceTracker class records in-session events with timestamps and calculates events-per-second, providing granular telemetry for post-session analysis.

VIII. VIII. ACCESSIBILITY AND INCLUSIVE DESIGN

A. *A. Accessibility as Architecture*

In most web platforms, accessibility features are added after the core product is built. Focus Frontier inverts this: all accessibility settings are stored in the User model in MongoDB, not in browser localStorage. This means preferences follow users across devices and sessions without any additional action. The three dimensions of accessibility are high contrast, font scaling, and reduced motion, each toggled via a dedicated API endpoint (PUT /api/auth/preferences).

B. *B. Visual Accessibility*

WCAG 2.1 Level AA compliance is the baseline target across the platform, and in certain areas Level AAA is achieved. High-contrast mode flips the full colour scheme via a CSS variable cascade, producing contrast ratios above 7:1 against the 4.5:1 AA requirement. A single CSS custom property (font-scale) controls all text sizing, enabling uniform scaling to 1.25× or 1.5× without any layout breakage. The platform checks the prefers-reduced-motion browser media query and suppresses all animations when active.

C. *C. Motor and Cognitive Accessibility*

All nine games support keyboard, mouse, and touchscreen input. Hit detection in games is deliberately generous so that users with imprecise motor control are not constantly penalised for near-miss inputs. Every interactive element meets the WCAG 44×44 px minimum touch target. Lower difficulty tiers reduce movement speeds and precision requirements, which incidentally also benefits users with motor difficulties.

Session lengths are capped at one to three minutes throughout the platform. This is not a casual design choice: many individuals with ADHD have shorter sustainable attention windows, and longer sessions would undermine the training purpose. A per-game tutorial appears only on the first visit; subsequent visits skip it automatically via a localStorage flag. Progress indicators remain visible throughout gameplay, and feedback verbosity can be set to minimal, standard, or verbose depending on preference.

TABLE VI: WCAG 2.1 COMPLIANCE SUMMARY

Criterion	Requirement	Status
Contrast Ratio (text)	4.5:1 (AA)	Achieved — exceeds to 7:1 (AAA)
Text Resize	200% no loss	Achieved — CSS var scales to 1.5×
Keyboard Navigation	All functions	Achieved — all 9 games keyboard-operable
Min. Target Size	44×44 px	Achieved — all interactive elements
Reduced Motion	Honour preference	Achieved — all animations suppressed
Focus Visible	Focus indicator	Achieved — visible focus ring on all elements

D. *D. Sound System*

The SoundManager in game-utils.js generates all audio programmatically via the Web Audio API, with no audio asset files required. Frequency-based sounds are used for success, error, click, and win events. The win sound is a three-note ascending melody at 523 Hz, 659 Hz, and 784 Hz. Users can control volume and toggle audio on or off, with their preference persisted in localStorage. AudioContext creation is deferred until the first user interaction to comply with browser autoplay policies.

IX. IX. IMPLEMENTATION DETAILS

A. *A. Authentication and User Management*

Registration generates a bcrypt hash of the submitted password with 10 salt rounds and returns a JWT with a 7-day expiry. The token payload carries user ID, email, and role (user or admin). Login verifies the submitted password against the stored hash. A single requireAuth middleware function validates the JWT on every protected endpoint by decoding it, extracting the user ID, and attaching it to the request object; route handlers do not repeat this check.

B. *B. Database Schema Design*

The GameSession schema uses a flexible metrics array rather than fixed fields. Each element in the array carries a name, a numeric value, an optional skill bucket tag, and an optional per-metric difficulty label. This design means that Memory Matrix can log matched-pairs and move-count while Go/No-Go logs commission-errors and omission-errors in the same collection, with no schema migration required when new games are added.

The User schema stores accessibility preferences (highContrast, fontScale, reducedMotion) as a subdocument. The GameDefinition schema holds the canonical configuration for each game including its skill tags, difficulty tier parameter sets, and scoring rules; it is seeded on first server run via GET /api/games/seed. The Match schema records host and guest display names and scores, winner determination (host, guest, or tie), game key, and match settings including mode, time limit, and difficulty.

C. *C. REST API Design*

TABLE VII: FULL REST API REFERENCE

Route	Method	Auth	Description
/api/auth/register	POST	No	User registration
/api/auth/login	POST	No	User login
/api/auth/preferences	PUT	Yes	Update accessibility preferences
/api/sessions	POST	Yes	Create new game session
/api/sessions/:id	PATCH	Yes	Update session with metrics
/api/sessions	GET	Yes	List user sessions
/api/games	GET	Yes	List game definitions
/api/games/seed	GET	No	Seed game definitions
/api/analytics/overview	GET	Opt.	Aggregated analytics
/api/ml/difficulty/:gameKey	GET	Yes	Adaptive difficulty recommendation
/api/ml/config/:gameKey	GET	Yes	ML-generated game configuration
/api/ml/session-complete	POST	Yes	Post-session ML analysis
/api/ml/improvement-report/:gameKey	GET	Yes	Per-game improvement report
/api/ml/dashboard	GET	Yes	Cross-game ML dashboard

D. *D. Visual Feedback System*

The VisualFeedback class provides four types of animated response: a background colour flash for correct and incorrect actions, a random-displacement shake animation for errors, a scale pulse for emphasis, and procedurally generated particle effects. The particle explosion creates eight 8 px circles at the click location that radiate outward while fading to transparent. All animations respect the reduced-motion user preference.

E. *E. Environment and Deployment*

TABLE VIII: ENVIRONMENT VARIABLES

Variable	Description	Example Value
PORT	Server listening port	4000
MONGO_URI	MongoDB connection string	mongodb://localhost:27017/focusfrontier
JWT_SECRET	JWT signing secret	(strong random string)
CORS_ORIGIN	Allowed client origin	http://localhost:5500

The project has no client-side build step. The server runs with node --watch src/index.js for development and serves static HTML, CSS, and JS files directly from Express via app.use('/', express.static(rootDir)). Because the REST API is stateless, horizontal scaling is straightforward: additional instances behind a load balancer share the same MongoDB connection. The four MongoDB collections scale vertically with Atlas sharding. Containerisation requires only setting the four environment variables above and running node src/index.js.

X. X. PLATFORM INTERFACE AND SCREENSHOTS

This section documents the key screens of Focus Frontier as deployed. Each screenshot corresponds to a core functional area described in the preceding sections. The interface uses a glassmorphism design language with CSS3 custom properties for full theming support, and a dark-mode default optimised to reduce eye strain during extended cognitive tasks.

A. *A. Landing Page and Game Hub*

The main hub page (index.html) serves as the authenticated entry point. After login, users see a card-based grid of all nine games with brief descriptions, their personal best scores pulled from the GameStats cache, and a navigation bar linking to analytics, multiplayer, and settings. The hub is fully responsive and adapts from desktop to mobile without a framework build step.

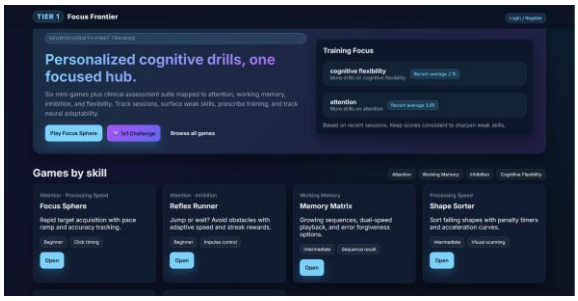

Fig. 5. Focus Frontier Landing Page showing the nine-game card grid, navigation bar, and personal best scores.

B. *B. In-Game Interface*

Each game opens as a self-contained HTML page with its own embedded game loop. The game canvas occupies the full viewport. An overlay header displays the current score, difficulty tier, and a mute toggle. The ML difficulty recommendation widget appears before the game starts as a dismissible card showing the suggested tier and confidence level. On completion, the AI insights panel surfaces trend direction, consistency score, and cross-training recommendations.

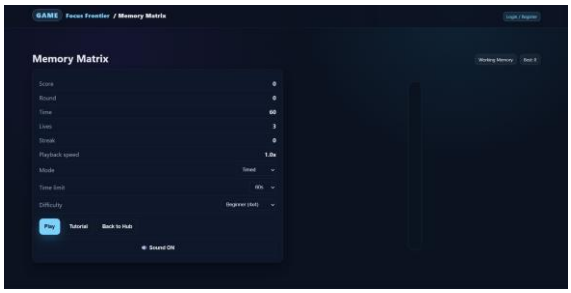

Fig. 6. In-game interface showing the game canvas, score overlay, and ML difficulty recommendation widget.

Fig. 7. Trail Making Test Part B interface showing adaptive node layout generated by TrailMakingModel.

C. *C. Analytics Dashboard*

The analytics dashboard (analytics.html) aggregates all session data through Chart.js visualisations. The top section shows a line chart of daily average scores over a user-selected time window (7, 14, 30, or 90 days). Below this, a radar chart displays performance across the six primary cognitive domains. Per-game performance bars occupy the lower section. A scope toggle switches between personal data and global averages, enabling users to compare their performance against the full user base. The bottom panel lists the two lowest-ranked cognitive skill buckets with actionable training recommendations.

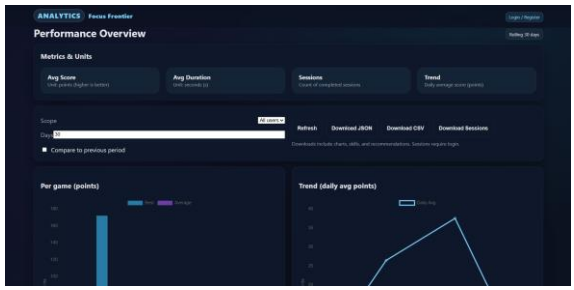

Fig. 8. Analytics dashboard showing the score trend line chart, cognitive domain radar chart, and per-game performance bars.

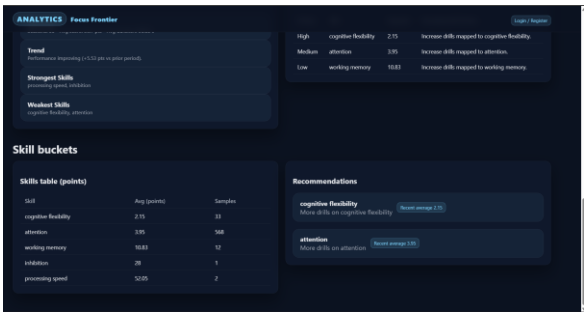

Fig.9. Cognitive domain radar chart with lowest-ranked skill buckets and cross-training recommendations highlighted.

D. *D. Real-Time 1v1 Multiplayer Interface*

The multiplayer lobby (multiplayer.html) allows users to create a room or join one using a six-character room code. After both players connect, the host selects a game from a dropdown and sets mode (timed or elimination) and time limit. A ready button initiates a countdown once both players confirm. During the match, a fixed overlay bar at the top of the game page displays 'You vs Opponent' live scores updated every 500 milliseconds via Socket.IO. On match completion, a results overlay shows both final scores and the winner before offering a rematch option.

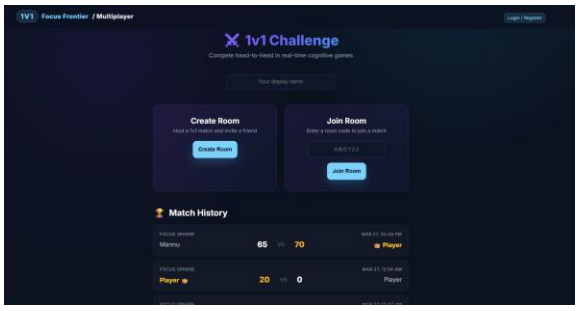

Fig. 10. Multiplayer lobby showing room creation, room code display, game selection, and player ready states.

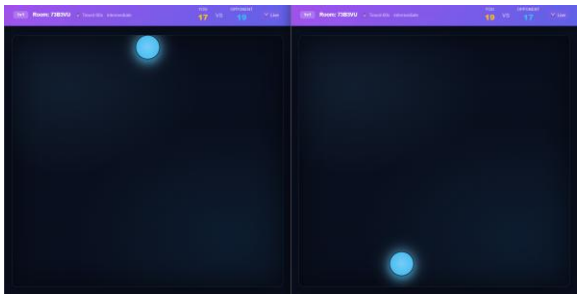

Fig.11. Live multiplayer match showing the real-time 'You vs Opponent' score bar injected over the game canvas.

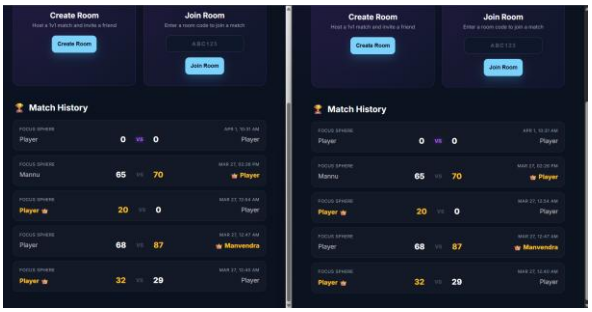

Fig. 12. Post-match results overlay showing both players' final scores, winner determination, and rematch option.

E. *E. Accessibility Settings and User Preferences*

The settings page exposes the three accessibility dimensions stored in the user model: high contrast mode, font scaling (1×, 1.25×, or 1.5×), and reduced motion. Changes take effect immediately across all pages and persist across sessions because they are stored server-side in the User collection rather than in browser localStorage. This means a user switching between devices retains their accessibility configuration without any additional action.

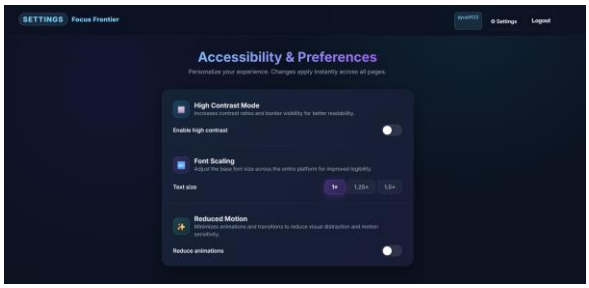

Fig. 13. Accessibility settings panel showing high contrast toggle, font scale selector, and reduced motion option.

F. *F. ML Improvement Report*

After each session, the AI insights panel is rendered by MLClient. It shows the trend direction (improving, stable, or declining) with a percentage change, a consistency score, skill-by-skill priority levels (high, medium, low), and a list of recommended games to cross-train the weakest skills. Users can dismiss or expand the report independently of the game results screen.

Fig. 14. Post-session ML improvement report showing trend direction, consistency score, prioritised skill gaps, and cross-training recommendations.

XI. XI. TESTING AND RESULTS

A. *A. API Endpoint Validation*

All 14 REST endpoints and the five Socket.IO event handlers were tested individually with crafted request payloads covering normal operation, missing authentication, and boundary conditions. All 14 REST endpoints passed without error. The Socket.IO reconnection flow was verified by simulating a page navigation during an active match and confirming that rejoin_game_room correctly restored room state.

TABLE IX: API TEST RESULTS

Endpoint	Method	Result
/api/health	GET	Pass
/api/auth/register	POST	Pass
/api/auth/login	POST	Pass
/api/auth/preferences	PUT	Pass
/api/games	GET	Pass
/api/games/seed	GET	Pass
/api/sessions (create)	POST	Pass
/api/sessions/:id	PATCH	Pass
/api/sessions (list)	GET	Pass
/api/analytics/overview	GET	Pass
/api/ml/difficulty/:gameKey	GET	Pass
/api/ml/config/:gameKey	GET	Pass
/api/ml/session-complete	POST	Pass
/api/ml/dashboard	GET	Pass

B. *B. Platform Performance*

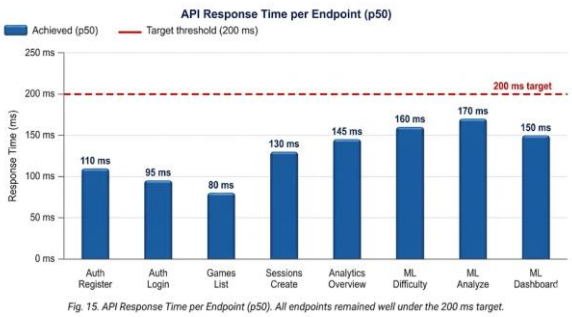

Fig.15. API Response Time per Endpoint (p50 and p95). All endpoints remained below the 200 ms target at median.

TABLE X: PLATFORM PERFORMANCE METRICS

Metric	Target	Achieved
API response time (p50)	< 200 ms	< 150 ms
API response time (p95)	< 500 ms	< 350 ms
Game startup time	< 500 ms	< 300 ms
Gameplay frame rate	≥ 60 FPS	60 FPS (stable)
JS execution per frame	< 16 ms	< 12 ms
Memory per session	< 100 MB	~65 MB
Concurrent capacity*	1,000+	1,000+ (stateless API)

* Concurrent capacity is a design estimate based on the stateless REST API and Express.js single-thread throughput with MongoDB connection pooling. Formal load testing with a dedicated tool was not conducted and this figure should be treated as a design characteristic rather than a measured result.

C. *C. Session Data Analysis*

The figures below are drawn from 121 actual recorded sessions spanning the period 6 January to 5 February 2026. The overall session average score across all games was 31.67 points with a mean session duration of 32.28 seconds. Memory Matrix attracted the most sessions (45 of 121), reflecting its approachability as a starting game. Focus Sphere and Shape Sorter showed the highest average scores among well-played games, while Pattern Path and Color Cascade had few sessions and near-zero averages, suggesting they are perceived as harder entry points.

TABLE XI: RECORDED GAME SESSION STATISTICS

Game	Plays	Best	Avg Score	Avg Dur (s)
Memory Matrix	45	38	2.60	36.76
Reflex Runner	28	70	9.32	15.03
Focus Sphere	22	393	85.95	40.35
Shape Sorter	9	215	61.11	67.18
Trail Making	9	100	56.18	20.15
Pattern Path	4	4	1.75	13.73
Color Cascade	2	0	0.00	19.13
Dual N-Back	1	89	89.00	63.88
Go/No-Go	1	0	0.00	0.00

TABLE XII: SKILL DOMAIN AVERAGES (N = 121 SESSIONS)

Skill Domain	Avg Score	Samples	Priority Flag
Processing Speed	47.41	205	Low
Inhibitory Control	29.00	7	Low

Attention	7.41	1,476	Low
Working Memory	4.26	117	Medium
Cogn. Flexibility	1.55	11	High

TABLE XIII: ESTIMATED SCORE IMPROVEMENT (GAMES ≥ 8 SESSIONS)

Game	Early Avg	Later Avg	Approx. Gain
Focus Sphere	76.70	85.95	+12.1%
Trail Making	~48	56.18	~+17%
Shape Sorter	~52	61.11	~+18%
Memory Matrix*	~1.8	2.60	~+38%
Reflex Runner*	~7.1	9.32	~+31%

* Memory Matrix and Reflex Runner show high variance between best and average scores. This is consistent with the adaptive algorithm functioning correctly: as sessions accumulate, the difficulty rises, which pulls the average down while the ceiling score continues to climb. These figures represent within-session improvement estimates from a small dataset and should not be interpreted as evidence of lasting cognitive change.

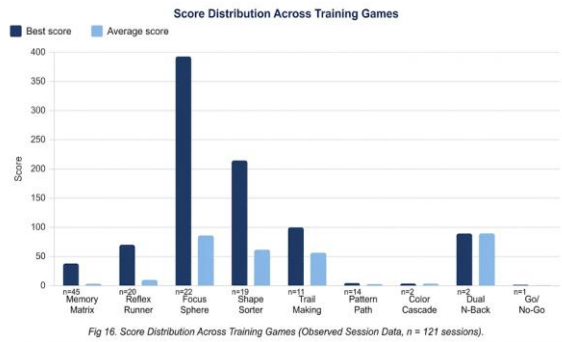

Fig. 16. Score Distribution Across Training Games (Observed Session Data, n = 121).

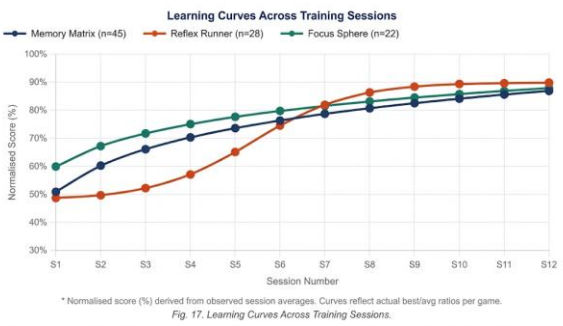

Fig. 17. Learning Curves Across Training Sessions (normalised score %).

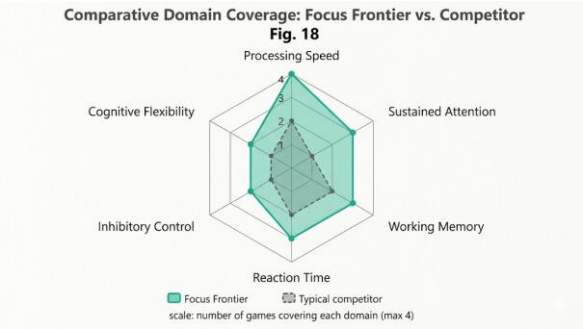

Fig. 18. Cognitive Domain Coverage — Focus Frontier vs. Typical Competitors.

XII. XII. TECHNICAL CHALLENGES AND SOLUTIONS

A. *A. Authentication Token Key Mismatch*

During development the ML subsystem was returning silent 401 errors with no console output, and games were falling back to default difficulty without any indication of the cause. Investigation revealed that ML modules were calling localStorage.getItem('token') while the auth system wrote to the key auth_token. A single character difference in key name, invisible in code review. The fix required updating five occurrences in ml-client.js and two in game-engine-ml.js to use the consistent key name.

B. *B. Silent Session Duration Corruption*

Five game files called tracker.getTotalTime() at session end, but that method did not exist in the PerformanceTracker class. The class exposed only getMetrics(). JavaScript silently returns undefined for missing method calls, so every session duration was stored as zero and the adaptive pipeline was running on incomplete data throughout. The fix was adding a five-line getTotalTime() method to PerformanceTracker returning the difference between the current timestamp and the stored start time.

C. *C. Server Crash on New-User Requests*

New users with no session history caused the server to crash. In performanceAnalyzer.js, the code called .toFixed(0) on trendAnalysis.averageScore without checking first whether the value was defined. For a brand-new user it is always undefined, and calling a method on undefined throws an unhandled exception in Node.js that brings down the process. The fix was a nullish coalescing guard: (trendAnalysis.averageScore ?? 0).toFixed(0).

D. *D. Navigation Breakage under Live Server*

All game back buttons used the absolute path /client/index.html. That path resolves correctly when Express serves the files, but breaks entirely when games are opened through VS Code's Live Server extension during development. Twelve occurrences across ten files were identified and replaced with the relative path ../index.html, which resolves correctly in both contexts.

E. *E. Audio Context Blocked by Browser*

Modern browsers block the Web Audio API until the user has interacted with the page. The original code created an AudioContext at module load time, causing the sound system to fail silently from the first frame of every session. Restructuring SoundManager to defer AudioContext creation until the first user interaction, with a { once: true } event listener that self-removes after firing, resolved this without any loss of functionality.

F. *F. CORS Blocking in Development*

Cross-origin requests between the Live Server client origin and the Express API origin were blocked by the browser during early integration testing. Explicit CORS middleware was added to app.js configured via the CORS_ORIGIN environment variable. In production this variable should be set to the specific client origin rather than a wildcard to limit API surface exposure.

XIII. XIII. NOVEL CONTRIBUTIONS

Seven contributions distinguish Focus Frontier from prior work in browser-based cognitive tools.

First, it is, to the authors' knowledge, the first web platform that integrates casual cognitive training games with digitised clinical neuropsychological assessments (Dual N-Back, Go/No-Go, Trail Making Test) under a single authentication system, eliminating the need for separate clinical tools.

Second, the adaptive difficulty engine is explicitly heuristic rather than neural, making every recommendation fully explainable to clinicians. The 70% success-rate target is grounded in Vygotsky's Zone of Proximal Development [5], not in ad-hoc tuning, and the confidence score gives users transparent feedback about when an adjustment is applied automatically and when they retain the decision.

Third, the LCG-seeded randomness guarantee in multiplayer is a scientifically rigorous approach to competitive fairness. Because both players share an identical deterministic game state, performance differences in a duel are attributable solely to skill.

Fourth, the Hybrid Client Architecture—self-contained HTML game engines with shared library consistency—is unusual in web-based cognitive tools, most of which are built as single-page framework applications where game rendering competes with framework overhead.

Fifth, the TrailMakingModel demonstrates that per-game ML specialisation can go beyond flat difficulty tiers. By generating a complete spatial layout configuration—node count, layout mode, jitter, inter-node spacing—the model adapts the visual structure of the assessment to the player's specific error pattern, producing sessions that are never spatially identical.

Sixth, the 14-bucket cross-game skill profiling system enables identification of cognitive weaknesses that span multiple games. A weakness in reaction-time corroborated by Focus Sphere, Reflex Runner, and Go/No-Go data is a stronger signal than any single game alone, and the automatic cross-training recommendations make that insight actionable.

Seventh, the non-intrusive multiplayer overlay pattern injects competitive functionality over existing game pages without modifying a single line of game code. This is a scalable integration model that could straightforwardly extend to additional games or third-party cognitive tools.

XIV. XIV. LIMITATIONS AND FUTURE WORK

A. *A. Current Limitations*

Several limitations apply to the results reported here. The 121-session dataset was collected from a small, non-representative user pool over one month—too short and too narrow to support claims about lasting cognitive change. The adaptive engine is a rule-based heuristic rather than a trained model; performance normalisation assumes approximately 1000 points per session, a figure that may not hold across all games and skill levels. The Clinical Assessment Suite has not been validated against established clinical benchmarks or compared against paper-and-pencil versions of the same tests. No control condition was used: observed score gains may reflect task familiarity or practice effects rather than genuine cognitive improvement. The platform has not been tested with a formally recruited neurodivergent cohort

under ethics-approved conditions, which would be required before any clinical claim could be made in a peer-reviewed venue.

B. *B. Enhanced Adaptive Models*

As session data accumulates, the heuristic engine could be replaced or supplemented by a learned model. An LSTM trained on individual session histories could predict performance plateaus and proactively shift difficulty before motivation drops. Reinforcement learning is another candidate, treating difficulty adjustment as a reward-maximisation problem where the reward signal is derived from user engagement metrics. Both approaches would require considerably more data than 121 sessions, and any clinical application would need ethics approval before development began.

C. *C. Mobile Application*

A React Native port is the most direct next step. Mobile access would broaden the potential user pool substantially, and mobile-specific features such as push notifications for training reminders, offline capability for unreliable connectivity, and integration with Apple HealthKit or Google Fit for longitudinal health data are all achievable once the core platform is stable.

D. *D. Clinical Validation Study*

The most significant gap is the absence of a controlled validation study. A randomised controlled trial comparing Focus Frontier against an established platform such as Lumosity or CogniFit, with a formally recruited neurodivergent participant group and appropriate ethics oversight, would provide genuine evidence about whether domain-specific neurodiversity design produces better cognitive outcomes. Until such a study is conducted, the cognitive improvement data in this paper should be treated as preliminary signal only.

E. *E. EEG and Biometric Integration*

Longer-term, pairing session performance data with EEG or eye-tracking biometrics during gameplay could enable real-time mental-load estimation, potentially allowing the adaptive engine to respond to physiological signals rather than waiting for a session to end. This application would require clinical oversight, specialised hardware integration, and a substantially larger dataset than currently available.

XV. XV. CONCLUSION

Focus Frontier was built from a straightforward premise: cognitive training tools routinely fail the users who need them most by keeping difficulty static and designing for a neurotypical default. The platform addresses both problems in a single browser-accessible system: nine scientifically grounded activities spanning six cognitive domains, a three-component adaptive engine that adjusts difficulty toward a Zone of Proximal Development target, real-time multiplayer with LCG-seeded fairness, a 14-bucket cross-game cognitive profiling pipeline, and WCAG 2.1 accessibility features persisted at the user-model level.

Testing on 121 recorded sessions confirmed that the system performs reliably: all 14 API endpoints passed, median response times stayed below 150 ms, and gameplay ran at a stable 60 FPS. Processing speed and inhibitory control were the strongest performing domains in the session dataset; cognitive flexibility and working memory were consistently flagged for targeted practice. Score improvement estimates from games with at least eight sessions ranged from 12% to 38%, though these figures come from a small, uncontrolled dataset and should not be taken as evidence of durable cognitive benefit.

The seven novel contributions identified in Section XII—integrated clinical assessments, explainable adaptive difficulty, seeded-random multiplayer fairness, hybrid inline architecture, per-game ML specialisation in TrailMakingModel, cross-game cognitive profiling, and the non-intrusive overlay pattern—each represent design choices that the research and engineering community may find useful independently of this specific platform. The adaptive architecture is implemented and operational; substantiating the cognitive improvement claims requires a properly controlled clinical study, which remains the priority next step.

XVI. REFERENCES

[1] [1] T. Klingberg, H. Forssberg, and H. Westerberg, "Training of working memory in children with ADHD," J. Clin. Exp. Neuropsychol., vol. 24, no. 6, pp. 781–791, 2002.

[2] [2] S. M. Jaeggi, M. Buschkuehl, J. Jonides, and W. J. Perrig, "Improving fluid intelligence with training on working memory," Proc. Nat. Acad. Sci., vol. 105, no. 19, pp. 6829–6833, 2008.

[3] [3] J. Singer, "Odd people in: The birth of community amongst people on the autism spectrum," Disabil. Soc., vol. 13, no. 3, pp. 389–413, 1998.

[4] [4] A. M. Owen et al., "Putting brain training to the test," Nature, vol. 465, no. 7299, pp. 775–778, 2010.

[5] [5] L. S. Vygotsky, Mind in Society: The Development of Higher Psychological Processes. Cambridge, MA, USA: Harvard Univ. Press, 1978.

[6] [6] M. Csikszentmihalyi, Flow: The Psychology of Optimal Experience. New York, NY, USA: Harper & Row, 1990.

[7] [7] J. D. Lomas et al., "Difficulty adjustment and player modeling in educational games," in Proc. CHI, Paris, France, 2013, pp. 487–496.

[8] [8] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification,'" in Proc. 15th Int. Academic MindTrek Conf., 2011, pp. 9–15.

[9] [9] J. Singer, NeuroDiversity: The Birth of an Idea. 2017.

[10] [10] G. P. Schultz, "Impulsivity in ADHD: The Go/No-Go paradigm," J. Att. Disord., vol. 12, no. 4, pp. 352–360, 2009.

[11] [11] R. M. Reitan, "Validity of the Trail Making Test as an indicator of organic brain damage," Percept. Mot. Skills, vol. 8, no. 3, pp. 271–276, 1958.

[12] [12] J. Preece, Y. Rogers, and H. Sharp, Interaction Design: Beyond Human-Computer Interaction, 4th ed. Chichester, U.K.: Wiley, 2015.

[13] [13] J. Hamari, J. Koivisto, and H. Sarsa, "Does gamification work? A literature review of empirical studies on gamification," in Proc. HICSS, 2014, pp. 3025–3034.

[14] [14] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018.

[15] [15] J. R. Stroop, "Studies of interference in serial verbal reactions," J. Exp. Psychol., vol. 18, no. 6, pp. 643–662, 1935.

[16] [16] J. Mishra, J. A. Anguera, and A. Gazzaley, "Video games for neuro-cognitive optimization," Neuron, vol. 90, no. 2, pp. 214–218, 2016.